

Отчёт лабораторная работа №14

Архитектура компьютеров и операционные системы

Никитенко Арина

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Задание 1. Реализация упрощённого механизма семафоров	6
2.2	Задание 2. Реализация команды map	9
2.3	Задание 3. Генерация случайной последовательности букв	11
3	Контрольные вопросы	13
4	Вывод	15

Список иллюстраций

2.1	Создание файла semafor.sh	6
2.2	Листинг semaphore.sh	7
2.3	Изменение прав доступа	8
2.4	Тестирование семафора терминал 1	8
2.5	Тестирование семафора терминал 2	9
2.6	Создание файла my_man.sh	9
2.7	Листинг my_man.sh	10
2.8	Изменение прав доступа	10
2.9	Тестирование my_man.sh	11
2.10	Создание файла random_letters.sh	11
2.11	Листинг random_letters.sh	12
2.12	Изменение прав доступа	12
2.13	Тестирование random_letters.sh	12

Список таблиц

1 Цель работы

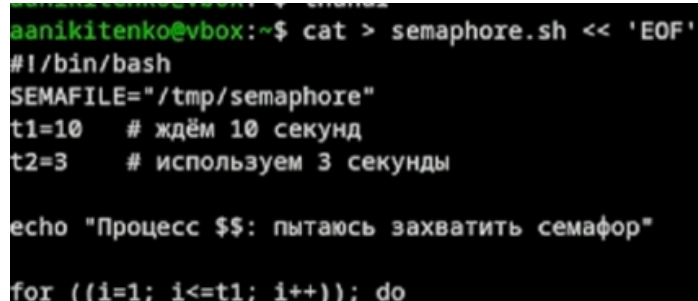
Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Выполнение лабораторной работы

2.1 Задание 1. Реализация упрощённого механизма семафоров

Разработаем скрипт `semafor.sh`, реализующий упрощённый механизм семафоров.

Создание файла `semafor.sh` (рис. 2.1):



```
aanikitenko@vbox:~$ cat > semafor.sh << 'EOF'
#!/bin/bash
SEMAFILE="/tmp/semaphore"
t1=10 # ждём 10 секунд
t2=3  # используем 3 секунды

echo "Процесс $$: пытаюсь захватить семафор"

for ((i=1; i<=t1; i++)); do
```

Рисунок 2.1: Создание файла `semafor.sh`

Листинг `semafor.sh` (рис. 2.2):

```
#!/bin/bash

LOCK_FILE="/tmp/semaphore.lock"
t1=5
```

```
while true; do
    if ln "$LOCK_FILE" "$LOCK_FILE.lock" 2>/dev/null; then
        echo "$$: [X][X][X][X] [X][X][X][X][X], [X][X][X][X][X][X] [X] [X][X][X][X] $t2 [X][X][X]"
        sleep $t2
        rm "$LOCK_FILE.lock"
        echo "$$: [X][X][X][X] [X][X][X][X][X][X]"
        break
    else
        echo "$$: [X][X][X][X] [X][X][X], [X][X][X][X] $t1 [X][X][X]"
        sleep $t1
    fi
done
```



7

```
chmod +x semafor.sh
```

```
rm -f "$SEMAFILE"
echo "Процесс $$: ресурс с
exit 0
fi
echo "Процесс $$: жду ($i сек
sleep 1
done
echo "Процесс $$: не дождался ресу
EOF
chmod +x semaphore.sh
```

Рисунок 2.3: Изменение прав доступа

Тестирование работы семафора (рис. 2.4-2.5):

```
# терминал 1
./semafor.sh

# терминал 2 (открыть терминал и запустить семафор)
./semafor.sh > /dev/tty1 &
```

```
chmod +x semaphore.sh
aanikitenko@vbox:~$ ./semaphore.sh
Процесс 168266: пытаюсь захватить семафор
Процесс 168266: ресурс свободен, захватываю
Процесс 168266: использую ресурс 3 сек
Процесс 168266: ресурс освобождён
```

Рисунок 2.4: Тестирование семафора терминал 1


```
anikitenko@vbox:~$ ./semaphore.sh > /dev/tty1 3>&1 &
```

Рисунок 2.5: Тестирование семафора терминал 2

2.2 Задание 2. Реализация команды man

Разработаем скрипт `my_man.sh`, который выводит справку по указанной команде из каталога `/usr/share/man/man1`.

Создание файла `my_man.sh` (рис. 2.6):

```
aanikitenko@vbox:~$ nano my_man
aanikitenko@vbox:~$ |
```

Рисунок 2.6: Создание файла `my_man.sh`

Листинг `my_man.sh` (рис. 2.7):

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "XXXXXXXXXXXXXXXX: $0 <XXXXXXXX>"
    exit 1
fi

CMD="$1"
MAN_FILE="/usr/share/man/man1/${CMD}.1.gz"

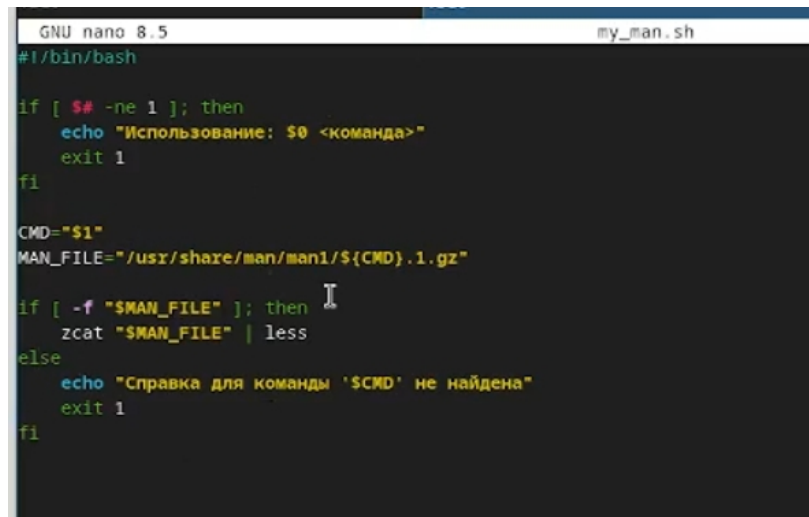
if [ -f "$MAN_FILE" ]; then
    zcat "$MAN_FILE" | less
```

```
else
```

```
    echo "Использование: $0 <команда>"
```

```
    exit 1
```

```
fi
```



```
GNU nano 8.5 my_man.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

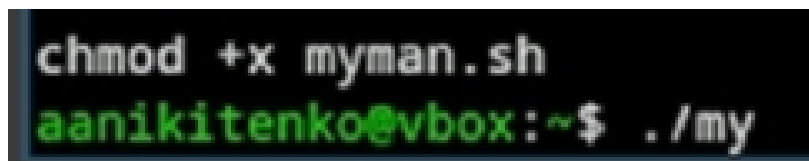
CMD="$1"
MAN_FILE="/usr/share/man/man1/${CMD}.1.gz"

if [ -f "$MAN_FILE" ]; then
    zcat "$MAN_FILE" | less
else
    echo "Справка для команды '$CMD' не найдена"
    exit 1
fi
```

Рисунок 2.7: Листинг my_man.sh

Делаем скрипт исполняемым (рис. 2.8):

```
chmod +x my_man.sh
```



```
chmod +x myman.sh
aanikitenko@vbox:~$ ./my
```

Рисунок 2.8: Изменение прав доступа

Тестирование работы my_man.sh (рис. 2.9):

```
./my_man.sh ls
```

```
./my_man.sh unknown_command
```

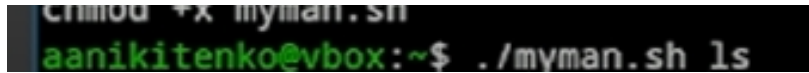


Рисунок 2.9: Тестирование my_man.sh

2.3 Задание 3. Генерация случайной последовательности букв

Разработаем скрипт `random_letters.sh`, генерирующий случайную последовательность букв латинского алфавита.

Создание файла **random_letters.sh** (рис. 2.10):



Рисунок 2.10: Создание файла random_letters.sh

Листинг **random_letters.sh** (рис. 2.11):

```
#!/bin/bash

LENGTH=${1:-20}

for ((i=0; i<LENGTH; i++)); do
    RAND=$((RANDOM % 26))
    LETTER=$(printf "\\$(printf '%03o' $((97 + RAND)))")
    echo -n "$LETTER"
done
echo
```



```

GNU nano 8.5 random_letters.sh
#!/bin/bash

LENGTH=${1:-20}

for ((i=0; i<LENGTH; i++)); do
    RAND=$((RANDOM % 26))
    LETTER=$(printf \\$(printf '%03o' $((97 + RAND))))
    echo -n "$LETTER"
done
echo

```

Рисунок 2.11: Листинг random_letters.sh

Делаем скрипт исполняемым (рис. 2.12):

```
chmod +x random_letters.sh
```



```

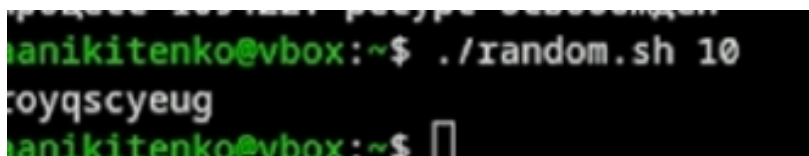
chmod +x random.sh

```

Рисунок 2.12: Изменение прав доступа

Тестирование работы **random_letters.sh** (рис. 2.13):

```
./random_letters.sh
./random_letters.sh 10
```



```

anikitenko@vbox:~$ ./random.sh 10
oyqscyeug
anikitenko@vbox:~$

```

Рисунок 2.13: Тестирование random_letters.sh

3 Контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке:

```
while [$1 != "exit"]
```

Ответ: пропущены пробелы после [и перед]. Правильно: `while [ "$1" != "exit" ]`

2. Как объединить (конкатенация) несколько строк в одну? Ответ: можно использовать оператор += или просто записать строки подряд:

```
str="Hello"
str+=" World"
# 
str="${str} World"
```

3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash? Ответ: seq генерирует последовательность чисел. Альтернативы:

```
for i in {1..10}; do echo $i; done
# 
for ((i=1; i<=10; i++)); do echo $i; done
```

4. Какой результат даст вычисление выражения `$((10/3))`? Ответ: 3 (целочисленное деление).

5. Укажите кратко основные отличия командной оболочки zsh от bash. Ответ: zsh имеет автодополнение с подсветкой, поддержку ассоциативных массивов, разделяемые истории между сеансами, более мощные возможности для работы с путями, улучшенную обработку глоббинга.

6. Проверьте, верен ли синтаксис данной конструкции:

```
for ((a=1; a <= LIMIT; a++))
```

Ответ: синтаксис верен.

7. Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки? Ответ:

Преимущества: простота автоматизации команд ОС, встроенные инструменты для работы с файлами и процессами, доступность в UNIX-системах.

Недостатки: медленная работа, ограниченная поддержка чисел с плавающей точкой, неудобный синтаксис для сложных структур, отсутствие объектно-ориентированного программирования.

4 Вывод

В ходе лабораторной работы были разработаны три скрипта:

- `semafor.sh` – реализация упрощённого механизма семафоров с использованием файловой блокировки.
- `my_man.sh` – аналог команды `man` для просмотра справки из `/usr/share/man/man1`.
- `random_letters.sh` – генерация случайной последовательности букв латинского алфавита с использованием `$RANDOM`.

Приобретены навыки межпроцессного взаимодействия через файловые блокировки, работы со сжатыми man-страницами (`zcat`), генерации случайных последовательностей, а также изучены особенности синтаксиса `bash`.